expensive for us to ignore how wasteful it would be performing them multiple times when we wouldn't need to.

Back to the vector sum above, if our vectors have billions of elements (or anyway too many to fit in a GPU's memory at once), then even this operation becomes quite expensive. Same if we are going to repeat the same divisions billions of times, when we could have just got away with a few hundreds of them: the impact on our applications' running time will be sensible.

Number crunching is not the only context in which optimizing your computation matters. If we move to web applications, for example, one of the costliest operations is certainly accessing a database, and even more if it involves iterating through a cursor to count or compute something.

It's not just terribly expensive (in terms of resources) and slow. Database cursors[2] might involve extensive (possibly even table- or DB-wide) locks[3] if they are not read-only, but even writing single rows can, in some DBs, require locking a page or a whole table.

Every time we lock some data, all read operations on those data will have to wait until the lock is released. If this is not handled carefully, a big load of write operations to the DB can put it on fire[4] slowing your application down and producing an inconvenient lag for your users, or even grind your DB to an halt, which in turn will cause all your HTTP calls to time out, and thus an outage of your website/application.